Preprocesamiento de datos

Unidad 2 · Valores faltantes, codificación, escalado, división de datos y pipelines

Imputación · One-hot · StandardScaler · Fuga de datos · Pipeline y ColumnTransformer

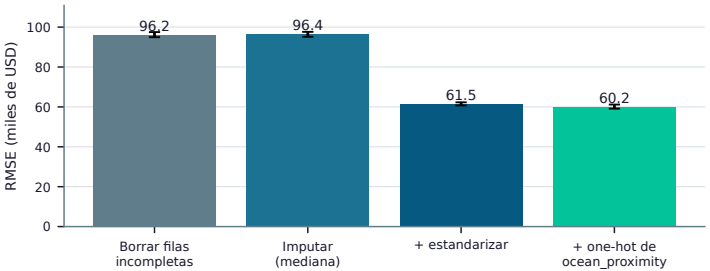
California Housing · pandas · scikit-learn

Plan de la unidad (≈45 min)

Qué veremos

- **A.** Valores faltantes: por qué faltan, borrar o imputar (media, mediana, KNN, iterativa)
- **B.** Variables categóricas: codificación entera, ordinal y *one-hot*
- **C.** Escalado: `StandardScaler`, `MinMaxScaler` y `RobustScaler`
- **D.** División entrenamiento / validación / prueba y **fuga de datos**
- **E.** `Pipeline` y `ColumnTransformer` de scikit-learn

¿Por qué preprocesar? El mismo modelo, resultados muy distintos



Mismo modelo (KNN, $k = 10$) y mismos datos; solo cambia el preprocesamiento. RMSE de validación cruzada de 5 particiones, en miles de USD (menor es mejor).

Lo que dice la gráfica

- Borrar o imputar casi no cambia el error (96.2 contra 96.4): hay solo 207 faltantes.
- **Estandarizar** lo baja de 96 a 61: el modelo estaba dominado por las variables de mayor escala.
- Codificar `ocean_proximity` lo baja a 60.

Idea de la unidad

Los modelos necesitan entradas **numéricas, completas y comparables**. Los datos crudos casi nunca lo son.

Nuestro conjunto de datos: California Housing (con faltantes y una categórica)

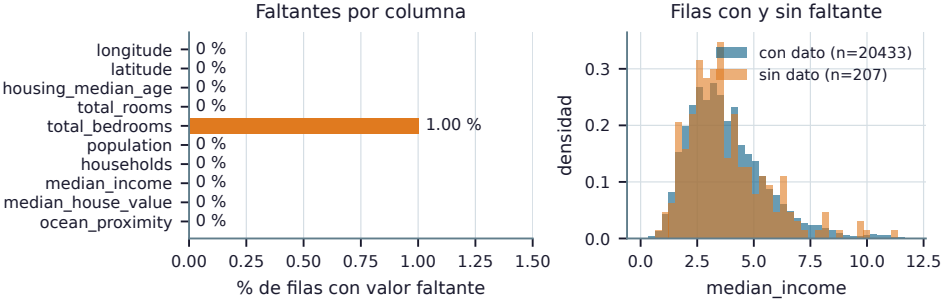
Variable	Qué describe (por distrito)
`longitude, latitude`	ubicación
`housing_median_age`	edad mediana de las viviendas
`total_rooms`	total de habitaciones
`total_bedrooms`	total de dormitorios **207 faltantes**
`population, households`	habitantes y hogares
`median_income`	ingreso mediano ($\times 10\,000$ USD)
`ocean_proximity`	**categórica**: cercanía al mar
`median_house_value`	**objetivo**: valor mediano (USD)

`ocean_proximity`	distritos
`<1H OCEAN`	9 136
`INLAND`	6 551
`NEAR OCEAN`	2 658
`NEAR BAY`	2 290
`ISLAND`	5

Esta unidad usa la versión del libro de A. Géron

$n = 20\,640$ distritos y 10 columnas. Es la misma fuente de la Unidad 1 en su forma original: columnas de totales (no promedios), una variable categórica y algunos faltantes, que son justo lo que necesitamos para practicar. La versión de scikit-learn ya viene limpia.

A1 · Detectar los valores faltantes



Izquierda: porcentaje de filas con faltante en cada columna. **Derecha**: comparamos el ingreso de los distritos con y sin dato en `total_bedrooms`; las dos distribuciones casi coinciden (ingreso medio 3.82 contra 3.87).

Cómo aparecen

- Como `NaN` (pandas lo marca con `isna()`).
- Disfrazados: `-999`, `"?"`, `""`, `0`. **Hay que buscarlos**: un 0 en una edad no es un dato.

Primeras preguntas

1. ¿Cuántos faltan y en qué columnas?
2. ¿Se parecen las filas con y sin faltante?
3. ¿Por qué faltan? (siguiente diapositiva)

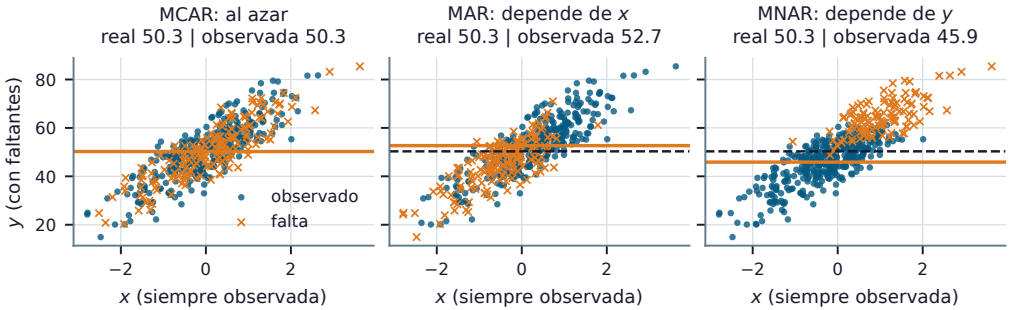
A2 · ¿Por qué faltan? Tres mecanismos

Mecanismo	Idea	Ejemplo
MCAR *faltan completamente al azar*	La probabilidad de faltar no depende de nada: ni de los datos observados ni del valor perdido.	Un sensor falla por una caída de energía; una fila se pierde al copiar el archivo.
MAR *faltan al azar, dado lo observado*	Depende de otra variable que *sí* tenemos, pero no del valor perdido.	Los más jóvenes (edad observada) responden menos a la pregunta sobre su ingreso.
MNAR *faltan no al azar*	Depende del *propio valor* que falta.	Quienes más ganan no declaran su ingreso.

Por qué importa

Con MCAR, borrar casi no sesga el análisis. Con MAR se puede corregir usando las otras variables (imputación basada en modelos). Con MNAR **ningún truco mecánico lo arregla**: hay que entender el proceso que genera los faltantes.

A3 · Cómo se ven los tres mecanismos



Datos simulados: x siempre se observa e y pierde el 30 % de sus valores. Azul = observado; × naranja = falta. Línea discontinua negra = media real de y; línea naranja = media de los valores observados.

MCAR. Los faltantes se reparten parejo: la media observada (50.3) coincide con la real (50.3).

MAR. Faltan sobre todo con x bajo, y como x e y se relacionan, la media observada se sesga (52.7). Se corrige usando x.

MNAR. Faltan los y altos: la media observada baja a 45.9 y **no hay** variable observada que lo remedie.

A4 · Borrar filas o columnas: lo más simple y su costo

Opciones

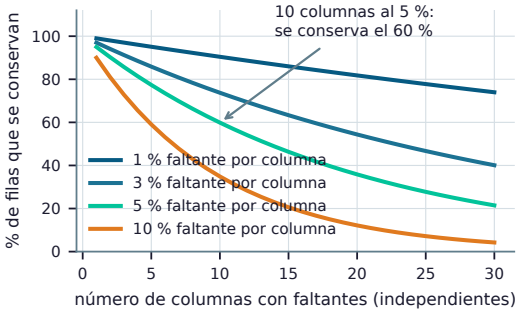
- **Borrar filas** con algún faltante (`dropna()`).
- **Borrar la columna** si falta casi toda.
- **Usar solo los datos disponibles** en cada cálculo (por pares).

Cuándo es razonable

Pocos faltantes (menos de $\approx 5\,\%$ de las filas), mecanismo cercano a MCAR y suficientes datos.

Si hay faltantes en varias columnas

Si cada columna pierde una fracción p de valores, de forma independiente, la fracción de filas completas es $(1-p)^k$ con k columnas: con $k = 10$ y $p = 5\,\%$ solo queda el 60 %; con $k = 20$, el 36 %.



En California Housing falta el 1 % de una sola columna: borrar cuesta 207 de 20 640 filas. Aquí es razonable; en general, no.

A5 · Imputación simple: sustituir por un valor típico

Estrategia	Valor	Cuándo
media	promedio	simétrica, sin atípicos
mediana	valor central	asimétrica o con atípicos
moda (`most_frequent`)	el más común	categóricas
constante	p. ej. 0 o "falta"	el faltante tiene significado

Ventaja

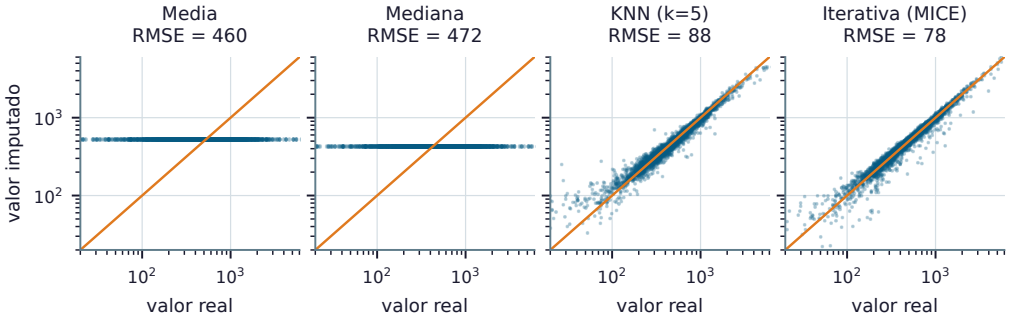
Conserva todas las filas y es muy rápida.

Costo: todas las filas reciben el mismo valor

- La **varianza** se encoge: se agregan muchos puntos iguales al centro.
- Las **correlaciones** se debilitan: el valor imputado no depende de las demás variables.
- Los faltantes ya no se distinguen de los datos reales.

Lo comprobamos en las dos diapositivas siguientes.

A6 · Un experimento: ocultar valores y compararlos con la verdad

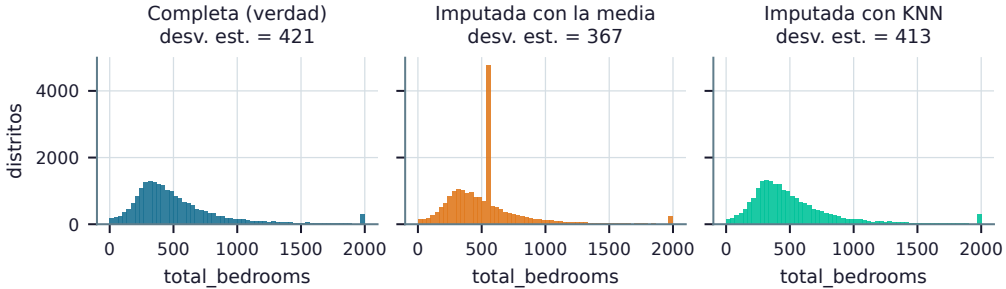


Ocultamos al azar el 20 % de `total_bedrooms` (4 134 valores) y los imputamos. Cada punto es un valor oculto: eje x, el valor real; eje y, el imputado. La diagonal naranja es la imputación perfecta. Escala logarítmica.

Media y mediana: todos los puntos caen en una *línea horizontal*: el mismo valor para distritos muy distintos. RMSE de 460 y 472 (la desviación estándar de la variable es 421).

KNN e iterativa: usan las otras columnas (hogares, habitaciones, población…) y los puntos siguen la diagonal. RMSE de 88 y 78.

A7 · Lo que la imputación le hace a la distribución



Histogramas de `total_bedrooms`: los datos completos (izquierda) y tras imputar el 20 % oculto con la media (centro) o con KNN (derecha). Arriba de cada panel, la desviación estándar.

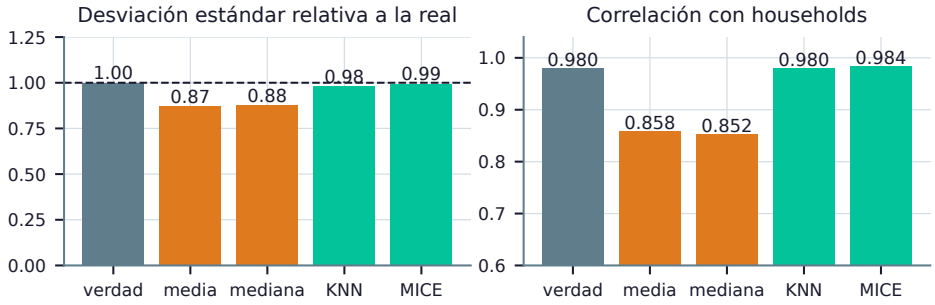
Media

Aparece un **pico enorme** en ≈ 537 (todas las imputaciones valen lo mismo). La desviación estándar baja de 421 a 367, un 13 % menos.

KNN

Conserva la forma de la distribución y su dispersión (desviación estándar 413): los valores imputados son distintos entre sí y razonables.

A7 · (continuación) Dispersión y correlación después de imputar



Izquierda: desviación estándar de `total_bedrooms` como fracción de la real (1 = igual). **Derecha**: correlación con `households`. La barra gris es el dato completo.

Qué se ve

- La correlación con `households` cae de **0.980** a **0.858** (media) y 0.852 (mediana), aunque la relación real no cambió.
- **KNN** e **iterativa** conservan la dispersión (0.98 y 0.99) y la correlación (0.980 y 0.984).

Conclusión

Con la media o la mediana, un análisis posterior (correlación, regresión) ve menos variabilidad y menos relación de la que hay. Imputar con las otras variables evita ese sesgo.

A8 · Imputar usando las otras variables: KNN

KNN (`KNNImputer`)

1. Para la fila con faltante, mide la distancia a las demás filas con las columnas que sí tiene.
2. Toma sus k vecinos más cercanos que tengan el dato.
3. Imputa el **promedio** de esos k valores.

Ojo: usa distancias, así que **necesita variables en la misma escala** (sección C).

Ventajas

Imputa valores que **existen** en los datos y conserva la dispersión. No supone una forma lineal entre las variables.

Cuidado

Depende de k y de la escala. Con muchas filas y columnas se vuelve lento, porque compara cada fila con todas las demás.

A8 · Imputación iterativa (1/2): predecir cada columna con las demás

La idea (`IterativeImputer`, tipo MICE)

Cada columna con faltantes se trata, **por turnos**, como la variable a predecir; las demás columnas son las predictoras.

1. **Relleno provisional** con la media de cada columna (solo para poder empezar).
2. **Elegir** una columna con faltantes, por ejemplo `total_bedrooms`.
3. **Entrenar** una regresión solo con las filas donde esa columna es real: $y =$ `total_bedrooms`, $X =$ las demás columnas.
4. **Predecir** las filas que faltaban y reemplazar el relleno por la predicción.
5. Pasar a la siguiente columna con faltantes: ya usa lo mejorado en la anterior.

Una pasada por todas las columnas es una **ronda**.

Ejemplo de juguete (6 filas)

fila	`households`	`total_bedrooms`
1	100	130
2	200	250
3	300	340
4	400	**falta**
5	**falta**	520
6	600	700

Ronda 1, columna `total_bedrooms`

Relleno inicial: `households` de la fila 5 $= 320$ (la media).

Con las 5 filas reales:
bedrooms ≈ 1.15 households $+ 38$.

Fila 4 (households $= 400$): **498.6**.

A8 · Imputación iterativa (2/2): repetir hasta que se estabilice

Los valores imputados por ronda

ronda	`total_bedrooms` (fila 4)	`households` (fila 5)
0 (media)	388.0	320.0
1	498.6	437.7
2	470.7	444.6
3	468.8	445.0
4	468.6	445.1
5	**468.6**	**445.1**

Regresión lineal simple en cada paso. Los cambios se vuelven mínimos y el método se detiene.

Siempre dentro del entrenamiento

Medias, medianas, vecinos y regresiones del imputador se **aprenden solo con el conjunto de entrenamiento** y luego se aplican a validación y prueba. Lo veremos en la sección D.

¿Por qué repetir?

La media del paso 0 es un mal relleno y contamina las predictoras. Al mejorar una columna, mejora la información para predecir la otra, y así sucesivamente hasta que ya no hay nada que corregir.

¿Cuándo se detiene?

Cuando los valores imputados casi no cambian entre rondas, o al llegar al máximo. En scikit-learn: `max_iter=10`, `tol=1e-3`. Suele bastar con pocas rondas.

Contra KNN

KNN **copia** el promedio de filas parecidas; la iterativa **aprende una fórmula** entre columnas y la aplica. Por eso aprovecha mejor las relaciones y no depende de distancias, pero es más lenta. Con los valores por defecto produce **una** imputación.

A9 · Indicador de faltante y cómo decidir

Agregar un indicador: ¿para qué?

Al imputar, el 435 se ve **igual que un dato real**: el modelo ya no sabe que ese valor se inventó. Si *el hecho de faltar* informa algo, esa señal se pierde. Por ejemplo, quien no declara su ingreso en una solicitud puede tener un riesgo distinto al de quien sí lo declara.

Solución: una columna 0/1 (`add_indicator=True`) que conserva el aviso:

`total_bedrooms`	**imputado**	**faltaba**
129	129	0
NaN	435	**1**
1 106	1 106	0

(imputado con la mediana, 435). Así el modelo puede darle un efecto propio a las filas que faltaban.

Costo: una columna más por variable con faltantes. Si faltar es puro azar (MCAR), solo agrega ruido.

Situación	**Acción**
$< 5\,\%$ de filas, MCAR	borrar filas o imputar con mediana
pocos, variable asimétrica	mediana
categórica	moda o categoría “falta”
variables relacionadas entre sí	KNN o iterativa
faltar es informativo (MNAR)	indicador + imputar
casi toda la columna falta	borrar la columna

B1 · Variables categóricas: ¿qué números les damos?

Los modelos solo entienden números

Una columna como `ocean_proximity` guarda texto. Hay que **codificarla**, y la forma de hacerlo *le dice algo al modelo* sobre las categorías.

Tipo	Qué tiene	Ejemplo
nominal	categorías sin orden	`INLAND`, `NEAR BAY`, color
ordinal	categorías con orden	talla S $<$ M $<$ L; nivel de estudios

Tres codificaciones

1. **Entera** (label): un número por categoría.
2. **Ordinal**: enteros que respetan un orden real.
3. **One-hot**: una columna 0/1 por categoría.

La pregunta clave: ¿el orden y la distancia entre números significan algo?

B2 · Codificación entera y ordinal



Si la variable es nominal

Los enteros inventan un **orden** (`ISLAND` "entre" `INLAND` y `NEAR BAY`) y **distancias** (`<1H OCEAN` está más cerca de `INLAND` que de `NEAR BAY`). Un modelo lineal o basado en distancias se lo cree.

Si la variable es ordinal

Se usa `OrdinalEncoder` con el **orden explícito**: `categories=[["S","M","L"]]` da $S=0$, $M=1$, $L=2$. Aquí la numeración sí tiene sentido (aunque la distancia entre niveles es una suposición).

Los modelos de árboles toleran la codificación entera mejor, porque solo comparan umbrales; los modelos lineales y de distancia, no.

B3 · Codificación one-hot: una columna por categoría

ocean_proximity	<1H OCEAN	INLAND	ISLAND	NEAR BAY	NEAR OCEAN
NEAR BAY	0	0	0	**1**	0
INLAND	0	**1**	0	0	0
<1H OCEAN	**1**	0	0	0	0
ISLAND	0	0	**1**	0	0

Ninguna categoría queda "más cerca" de otra: todas están a la misma distancia $\sqrt{2}$.

Detalles prácticos

- **Una columna sobra.** Las columnas suman siempre 1: si cuatro valen 0, la quinta vale 1. En regresión lineal esto causa la "trampa de la variable ficticia" y se resuelve con `drop="first"`.
- **Categoría nueva en prueba.** Con `handle_unknown="ignore"` queda toda en ceros en vez de provocar un error.
- **Cuántas columnas.** c categorías dan c columnas, o $c-1$ con `drop="first"` (aquí, 5 o 4).

Ambos detalles, en la siguiente diapositiva.

B3 · (continuación) Los dos detalles: la columna que sobra y la categoría nueva

1 · La trampa de la variable ficticia

ocean_proximity	<1H OCEAN	INLAND	ISLAND	NEAR BAY	NEAR OCEAN	suma
NEAR BAY	0	0	0	1	0	**1**
INLAND	0	1	0	0	0	**1**
<1H OCEAN	1	0	0	0	0	**1**
NEAR OCEAN	0	0	0	0	1	**1**

Cada fila suma 1, igual que la columna de unos (el intercepto). Una columna se obtiene de las otras y la matriz $X^\top X$ **no se puede invertir**: no hay una solución única.

Solución: `drop="first"` elimina `<1H OCEAN`, que queda como **referencia** (fila de ceros). Los coeficientes se leen respecto a ella. Con Ridge, Lasso, árboles o redes no hace falta quitarla.

2 · Una categoría que no vimos

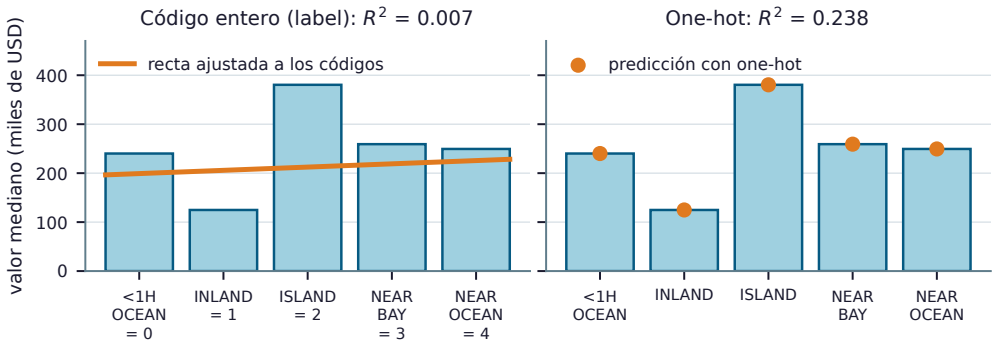
El codificador aprende las categorías **solo con el entrenamiento** (`fit`). Si en prueba llega `COASTAL`:

- Por defecto: **error**.
- Con `handle_unknown="ignore"`: la fila queda en `[0, 0, 0, 0, 0]`, es decir, "no es ninguna de las conocidas".

Cuidado al combinar las dos opciones

Con `drop="first"`, la fila de ceros ya significa "categoría de referencia". Una categoría desconocida se confunde entonces con ella. Si esto importa, conviene no usar `drop` (con regularización o árboles) o revisar las categorías nuevas antes de codificar.

B4 · ¿Importa la codificación? Entera contra one-hot



Barras: valor mediano de la vivienda promedio de cada categoría (miles de USD). Regresión lineal sobre `ocean_proximity` con las dos codificaciones.

Código entero (alfabético): la regresión ajusta *una recta* sobre 0, 1, 2, 3, 4. Las categorías no están ordenadas por precio, así que la recta casi no explica nada: $R^2 = 0.007$.

One-hot: cada categoría tiene su propio coeficiente y la predicción es su promedio. $R^2 = 0.238$. (`ISLAND` tiene solo 5 distritos: su promedio es poco confiable.)

B5 · Muchas categorías y categorías raras

El problema

Un código postal o una ciudad pueden tener miles de valores: one-hot crea miles de columnas, la mayoría casi vacías, y muchas categorías casi sin ejemplos (como `ISLAND`, con 5 de 20 640).

Primera salida: agrupar las raras

Juntar las categorías con pocos casos en "otra" (`min_frequency` en `OneHotEncoder`). Si aun así quedan demasiadas, se usa una de estas alternativas, que dejan **pocas columnas**:

Alternativa	Qué número pone	Ejemplo con `ocean_proximity`	Cuidado
frecuencia	la proporción de filas de esa categoría	`INLAND` $\to 0.32$ (6 551 de 20 640); `ISLAND` $\to 0.0002$	categorías distintas con la misma frecuencia se confunden
objetivo (*target encoding*)	el promedio del valor a predecir en esa categoría	`INLAND` $\to\approx 125$; `ISLAND` $\to\approx 380$ (miles de USD, de la figura anterior)	**Riesgo de fuga**: usa la respuesta. Calcularlo solo con el entrenamiento y, mejor, con validación cruzada interna. Las categorías raras salen poco confiables
hashing	una función hash asigna cada categoría a una de m columnas fijas	con $m = 3$, cada categoría cae en una de tres columnas	dos categorías pueden **chocar** en la misma columna y perder identidad; no es interpretable

C1 · ¿Por qué escalar? Variables con unidades muy distintas

Variable	mínimo	máximo
median_income	0.5	15
housing_median_age	1	52
latitude	32.5	42.0
total_rooms	2	39 320
population	3	35 682

El problema

Una variable medida en miles pesa más en una distancia, un gradiente o una penalización que otra medida en unidades, *solo por sus unidades*.

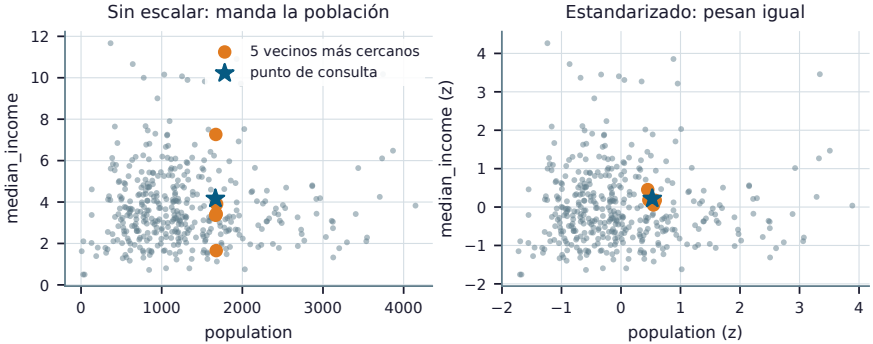
Sí necesitan escalado

- **Basados en distancias**: KNN, k-means, SVM.
- **Basados en gradiente**: regresión con descenso de gradiente, redes neuronales (converge mucho más rápido).
- **Regularizados**: Ridge, Lasso (la penalización depende de la escala de cada coeficiente).
- PCA.

No lo necesitan

Árboles de decisión, bosques aleatorios y *boosting*: solo comparan umbrales dentro de cada variable.

C2 · La distancia sin escalar: manda la variable más grande



Cómo leerla

Cada punto es un distrito. La estrella es el punto de consulta y los puntos naranja sus 5 vecinos más cercanos.

Sin escalar

`population` varía en miles y `median_income` en unidades: la distancia casi solo mira la población. Los vecinos difieren en ingreso hasta 5.6 unidades.

Estandarizado

Los vecinos son cercanos en *ambas* variables (diferencia máxima de ingreso: 0.4 desviaciones estándar).

C3 · Tres escaladores

Escalador	Fórmula	Qué aprende (`fit`)	Resultado
`StandardScaler`	$x' = \dfrac{x - \mu}{\sigma}$	media μ y desviación estándar σ	media 0, desv. est. 1; sin límites
`MinMaxScaler`	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$	mínimo y máximo	todo en $[0, 1]$
`RobustScaler`	$x' = \dfrac{x - \text{mediana}}{Q_3 - Q_1}$	mediana y cuartiles	mediana 0, IQR 1; sin límites

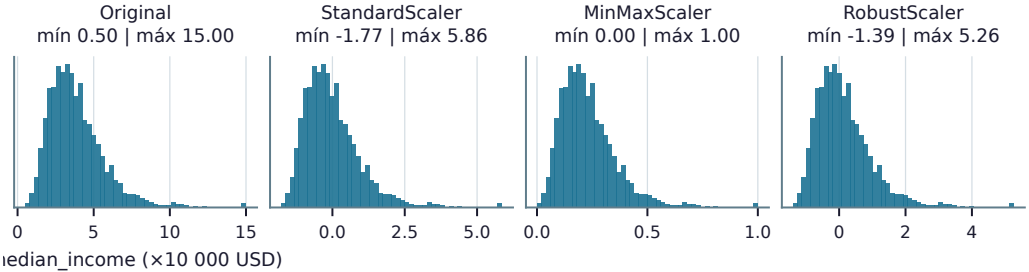
Idea común

Restar un valor **de centro** y dividir entre una medida de **dispersión**. Cada escalador elige esas dos medidas de forma distinta.

Los parámetros se aprenden de los datos

μ, σ, mínimo, máximo, mediana y cuartiles se calculan con el **entrenamiento** y se reutilizan en validación y prueba.

C4 · Escalar cambia la escala, no la forma

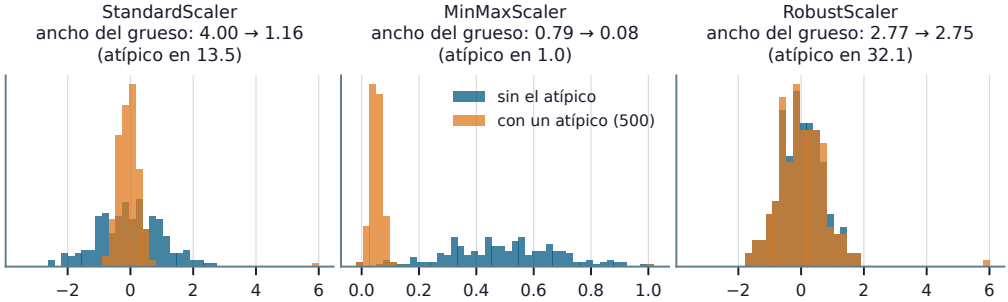


Histograma de `median_income` antes y después de cada escalador. La silueta (asimetría a la derecha, cola larga) es **la misma** en los cuatro; solo cambian los números del eje: Standard va de -1.77 a 5.86; MinMax, de 0 a 1; Robust, de -1.39 a 5.26.

Consecuencia

Escalar **no arregla** la asimetría ni quita atípicos. Para eso se usan transformaciones como el logaritmo (Unidad 1).

C5 · ¿Qué pasa con un dato atípico? Experimento con 200 datos



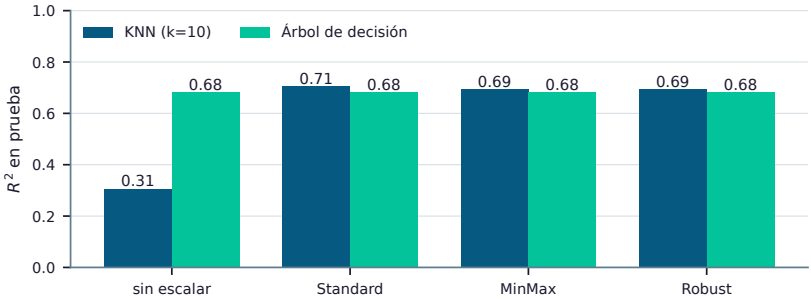
200 datos $N(50, 10)$ (azul) y los mismos 200 más **un solo** dato en 500 (naranja). El "ancho del grueso" es 4 desviaciones estándar de los 200 datos originales después de escalar.

Standard: el atípico infla σ (de 10 a ≈ 34); el grueso se encoge de ancho 4.0 a 1.2.

MinMax: el atípico fija el máximo; el grueso queda comprimido en ≈ 0.08 del intervalo $[0, 1]$.

Robust: usa mediana y cuartiles, que casi no se mueven: el grueso mantiene su ancho (2.77 contra 2.75).

C6 · ¿Y en un modelo? Escalar sí o no, y cuál



R^2 en el conjunto de prueba al predecir `median_house_value` con las 8 variables numéricas. KNN ($k = 10$) y árbol de profundidad 8.

Qué muestra

- **KNN**: R^2 pasa de 0.31 a 0.71 al estandarizar. Los tres escaladores funcionan (0.69–0.71).
- **Árbol**: 0.68 en los cuatro casos; no le afecta.

Cómo elegir

- `StandardScaler`: opción por defecto.
- `MinMaxScaler`: cuando se necesita un rango fijo (imágenes, redes) y no hay atípicos.
- `RobustScaler`: con atípicos o colas pesadas.

D1 · Para qué dividir: entrenamiento, validación y prueba



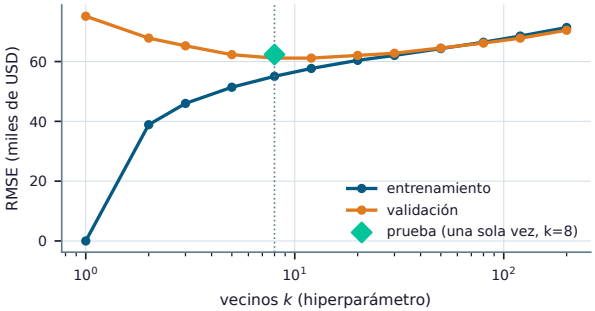
La analogía del examen

Entrenamiento = problemas de práctica. Validación = examen de ensayo (con él decides cómo estudiar). **Prueba = examen final**, que no puedes ver antes. Si lo ves y corriges, la calificación deja de medir lo que sabes.

Por qué no basta el error de entrenamiento

Un modelo puede memorizar el entrenamiento (error casi 0) y fallar con datos nuevos. Solo datos que el modelo *no vio* miden su capacidad de generalizar. Las proporciones (60/20/20, 70/15/15, 80/10/10) dependen del tamaño de los datos.

D2 · El conjunto de validación elige; el de prueba confirma



Qué muestra

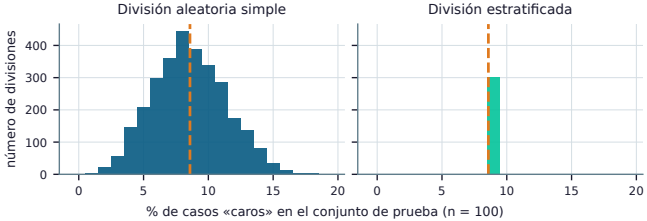
KNN con distintos k (12 384 / 4 128 / 4 128 distritos en entrenamiento, validación y prueba).

- **Entrenamiento**: con $k = 1$ el error es 0 (memoriza) y crece al suavizar.
- **Validación**: error alto con $k = 1$ (75.2), mínimo en $k = 8$ (61.1) y sube de nuevo.
- **Prueba**, una sola vez en $k = 8$: 62.3, cercano al de validación.

Regla

Si usas el conjunto de prueba para decidir algo (un k, un modelo, un umbral), ya no es de prueba: pasó a ser de validación.

D3 · Cómo dividir: aleatorio, estratificado, por grupos o por tiempo



8.6 % de los distritos son "caros" (valor $\geq 400\,000$ USD). Con 3 000 divisiones aleatorias, el % en un conjunto de prueba de 100 distritos varía entre 1 y 20; con estratificación siempre es ≈ 9.

División	**Cuándo**
aleatoria	datos independientes y clases balanceadas
estratificada (`stratify=y`)	clases raras: conserva la proporción en cada conjunto
por grupos	varias filas de la misma persona, paciente o cliente: el grupo no se parte. *Ejemplo:* 5 radiografías por paciente; si se reparten entre entrenamiento y prueba, el modelo reconoce al paciente y no la enfermedad (`GroupKFold`, `groups=id_paciente`)
temporal	series de tiempo: entrenar con el pasado, probar con el futuro

D4 · Validación cruzada de k particiones



Cómo funciona

Se parten los datos de entrenamiento en k bloques. En cada ronda, un bloque valida y los otros $k-1$ entrenan. El error estimado es el **promedio** de las k rondas:

$$\widehat{E} = \frac{1}{k} \sum_{j=1}^{k} E_j$$

Ventajas y costo

- Usa todos los datos para entrenar y validar.
- Da también una *variabilidad* del error (desviación estándar entre rondas).
- Cuesta k entrenamientos. Con $k = 5$ o 10 es lo habitual.
- El conjunto de prueba sigue aparte.

D5 · Fuga de datos: cuando el modelo “ve” lo que no debe

Definición

Hay **fuga** (*data leakage*) cuando, al entrenar, el modelo usa información que no tendrá al hacer predicciones reales. El resultado es una evaluación **demasiado optimista** que se desmorona en producción.

Tipo	Ejemplo	Cómo evitarla
Preprocesar antes de dividir	calcular la media, la mediana, σ o los atributos elegidos con *todos* los datos	dividir primero; `fit` solo con el entrenamiento
Fuga del objetivo	una columna que ya contiene la respuesta, p. ej. “días de atraso” para predecir si habrá mora	preguntar: ¿tendría este dato al momento de predecir?
Fuga temporal	mezclar el futuro con el pasado en una serie de tiempo	división temporal
Duplicados	la misma fila (o la misma persona) en entrenamiento y prueba	eliminar duplicados; dividir por grupos

D5 · (continuación) Preprocesar antes de dividir: cómo se filtra la información

Dos órdenes posibles

Mal: (1) calcular media, σ, medianas o atributos con **todos** los datos $\rightarrow$ (2) dividir $\rightarrow$ (3) entrenar y evaluar.

Bien: (1) dividir $\rightarrow$ (2) calcular los estadísticos solo con el entrenamiento (`fit`) $\rightarrow$ (3) transformar el entrenamiento y entrenar $\rightarrow$ (4) transformar validación y prueba **con esos mismos estadísticos**.

¿Dónde está la trampa?

Esas estadísticas se calculan con filas de prueba. Los datos de prueba ya **influyeron** en cómo se ven los de entrenamiento, así que dejan de ser datos nuevos y la evaluación sale optimista.

Ejemplo: estandarizar x

Entrenamiento: $1, 2, \ldots, 8$. Prueba: 9 y 40 (un dato atípico).

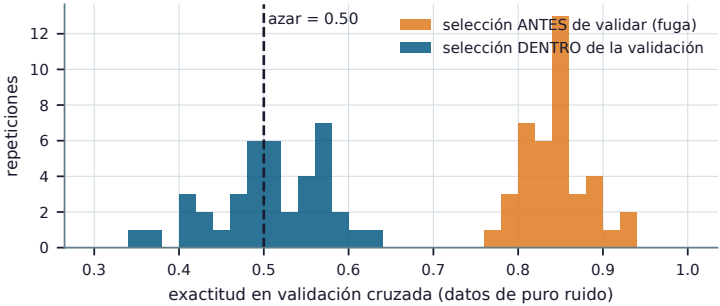
	μ	σ	$z(9)$	$z(40)$
con todos	8.5	10.8	0.05	2.9
con el entrenamiento	4.5	2.3	2.0	**15.5**

Con todos los datos, el 40 **infló** σ y parece casi normal ($z = 2.9$). Con el entrenamiento se ve lo lejos que está, que es lo que pasará con un dato así en producción.

¿Cuánto daño hace?

Depende de lo que se calcule. Estandarizar o imputar con la media suele sesgar poco. **Elegir atributos con todos los datos** puede dar resultados ilusorios, como veremos en D6 con puro ruido.

D6 · Un experimento: encontrar patrones en puro ruido



El experimento

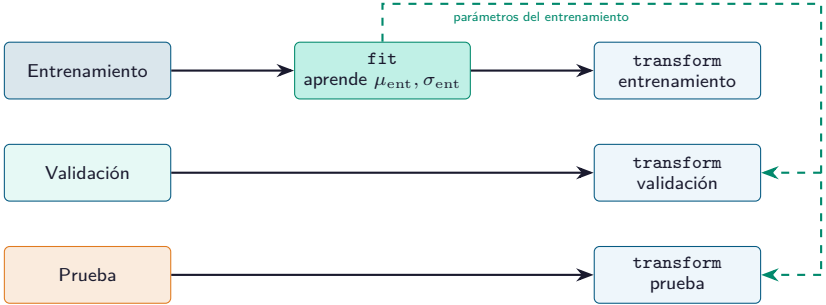
100 filas, **2 000 atributos de ruido puro** y una etiqueta 0/1 al azar: **no hay nada que aprender**. Repetimos 40 veces:

1. Elegir los 20 atributos más "relacionados" con la etiqueta usando *todos* los datos y *después* validar con validación cruzada.
2. Hacer lo mismo pero eligiendo los atributos *dentro* de cada partición de entrenamiento.

Resultado

Con fuga: exactitud media de **0.84**, aparentemente excelente. Sin fuga: **0.51**, es decir, azar, que es lo correcto.

D7 · Regla de oro: `fit` con el entrenamiento, `transform` con todos



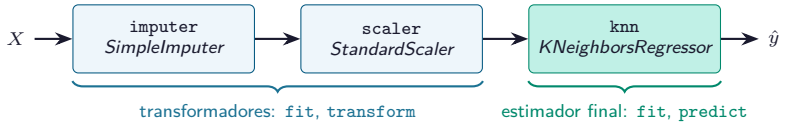
$$z_{\text{prueba}} = \frac{x_{\text{prueba}} - \mu_{\textbf{ent}}}{\sigma_{\textbf{ent}}} \quad (\text{no } \mu_{\text{prueba}})$$

Los datos de validación y prueba se transforman con los parámetros del entrenamiento, aunque sus propias medias sean otras.

Errores comunes

- `fit_transform` sobre todo el conjunto antes de dividir.
- Llamar `fit` otra vez sobre la prueba.
- Elegir atributos o imputar con todos los datos.
- Hacer validación cruzada sobre datos ya preprocesados.

E1 · `Pipeline`: encadenar pasos y un modelo



Paso	Qué aprende en `fit`	Qué hace después
`imputer`	la mediana de cada columna	`transform`: rellena los faltantes
`scaler`	la media y σ de cada columna	`transform`: estandariza
`knn`	guarda los datos de entrenamiento (sus vecinos)	`predict`: estima $\hat{y}$

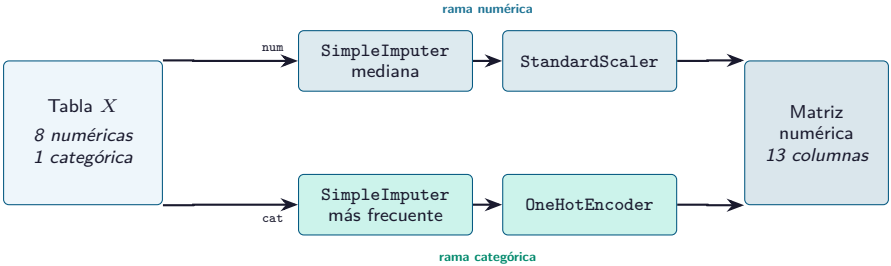
Qué hace `pipe.fit(X_ent, y_ent)`

Ajusta el imputador con `X_ent`, transforma, ajusta el escalador con lo transformado, transforma y entrena el modelo. **Todo con el entrenamiento.**

Qué hace `pipe.predict(X_nuevo)`

Aplica a `X_nuevo` el imputador y el escalador *ya ajustados*, y le pasa el resultado al modelo. Sin `fit` nuevo: no hay fuga.

E2 · `ColumnTransformer`: un tratamiento para cada tipo de columna



Cómo se declara

Una lista de tuplas `(nombre, transformador, columnas)`: `("num", num_pipe, num_cols)` y `("cat", cat_pipe, cat_cols)`. Cada rama puede ser a su vez un `Pipeline`.

Salida

8 numéricas + 5 columnas one-hot = **13 columnas**. Se pueden listar con `get_feature_names_out()`.

E3 · El preprocesador completo en código

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num = ["longitude", "latitude", "housing_median_age", "total_rooms",
       "total_bedrooms", "population", "households", "median_income"]
cat = ["ocean_proximity"]

num_pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
                     ("scaler",  StandardScaler())])
cat_pipe = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                     ("onehot",  OneHotEncoder(handle_unknown="ignore"))])

prep = ColumnTransformer([("num", num_pipe, num),
                          ("cat", cat_pipe, cat)])
```

E4 · Preprocesamiento + modelo: ajustar y evaluar sin fuga

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split, cross_val_score

model = Pipeline([("prep", prep),
                  ("knn",  KNeighborsRegressor(n_neighbors=10))])

X = df.drop(columns="median_house_value")
y = df["median_house_value"]
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2,
                                              random_state=0)

# validacion cruzada: el preprocesamiento se reajusta en cada ronda
rmse = -cross_val_score(model, X_ent, y_ent, cv=5,
                        scoring="neg_root_mean_squared_error")
print(rmse.mean(), rmse.std())        # 60 493  +-  1 576

model.fit(X_ent, y_ent)               # aprende todo SOLO con X_ent
print(model.score(X_pru, y_pru))      # R^2 en prueba: 0.715
```

Un solo objeto (`model`) contiene todo el procesamiento: se puede guardar, reutilizar con datos nuevos y pasar a `cross_val_score` sin riesgo de fuga.

E5 · Buscar hiperparámetros del modelo *y* del preprocesamiento

```python
from sklearn.model_selection import GridSearchCV

param_grid = {
    "knn__n_neighbors": [5, 10, 20],                  # del modelo
    "prep__num__imputer__strategy": ["mean", "median"],  # del imputador
}
busqueda = GridSearchCV(model, param_grid, cv=5,
                        scoring="neg_root_mean_squared_error")
busqueda.fit(X_ent, y_ent)

print(busqueda.best_params_)
# {'knn__n_neighbors': 10, 'prep__num__imputer__strategy': 'median'}
print(-busqueda.best_score_)           # RMSE de validacion: 60 493
```

Convención de nombres: `paso__subpaso__parametro`, con doble guion bajo. Aquí: `prep` → `num` → `imputer` → `strategy`.

En cada partición, el imputador y el escalador se reajustan con el pedazo de entrenamiento: **la búsqueda tampoco tiene fuga**.

Receta de preprocesamiento en siete pasos

1. **Dividir primero**: entrenamiento / validación / prueba (estratificado si hace falta).
2. **Explorar** solo el entrenamiento: faltantes, tipos, atípicos.
3. **Faltantes**: entender el mecanismo; borrar o imputar (mediana, KNN, iterativa) y, si importa, agregar un indicador.
4. **Categóricas**: one-hot (nominales), ordinal con orden explícito (ordinales).
5. **Escalar** si el modelo usa distancias, gradientes o regularización.
6. **Empaquetar todo** en `Pipeline` + `ColumnTransformer`.
7. **Validar** con validación cruzada y tocar la prueba **una vez**.

Las tres ideas de la unidad

1. Cada decisión (cómo imputar, cómo codificar, cómo escalar) **cambia lo que el modelo aprende**.
2. Todo lo que se *aprende de los datos* debe aprenderse **solo del entrenamiento**.
3. El `Pipeline` es la herramienta que hace cumplir (2) automáticamente.

Actividad: construir un pipeline de preprocesamiento

Notebook de la actividad (trabajo individual)

1. Cargar California Housing y **cuantificar los faltantes**.
2. Dividir en entrenamiento y prueba (con estratificación).
3. Comparar **estrategias de imputación** sobre valores ocultados.
4. Codificar `ocean_proximity` con enteros y con one-hot, y comparar.
5. Comparar `StandardScaler`, `MinMaxScaler` y `RobustScaler` con KNN.
6. **Provocar y medir una fuga** de datos, y corregirla con un `Pipeline`.
7. Armar el `ColumnTransformer` completo y evaluarlo con validación cruzada.

Entregable

Una tabla de comparación:

estrategia | **RMSE de validación** | **comentario**

Formato: individual. Al terminar, sube el notebook ejecutado y la tabla.

Tiempo: lo que quede de sesión.



¿Preguntas?

¡Pasemos a la actividad con California Housing!
